

TD1 : Constraint Programming — University Timetabling

This exercise sheet illustrates Constraint Programming on a realistic combinatorial problem : building a **course timetable** for a small university department. As in Exercise Sheet 1, you will model the problem on paper, run propagation and search algorithms by hand on small instances, and then implement and extend the model with Choco-Solver.

Running scenario. The Computer Science department must schedule 6 courses over 2 days, with 3 one-hour time slots per day (so 6 slots in total, numbered 1 to 6 : slots 1–3 on Monday, slots 4–6 on Tuesday). Two student groups, L3A (35 students) and L3B (20 students), share the department's two rooms : R1 (amphitheatre, capacity 60) and R2 (classroom, capacity 25). Each course lasts exactly one slot.

Course	Name	Teacher	Group(s)	Attendance
C_1	Algorithmique	T_1	L3A	35
C_2	Bases de données	T_2	L3A	35
C_3	Réseaux	T_1	L3B	20
C_4	Intelligence Artificielle	T_3	L3A $\cup$ L3B	55
C_5	Anglais	T_4	L3A	35
C_6	Probabilités	T_2	L3B	20

A room can only host *one* course per slot, and can only host a course whose attendance does not exceed its capacity (so C_4 , and any course attended by L3A, needs R1 ; a course attended only by L3B may use either room).

Exercise 1

We first model the scenario above as a constraint network.

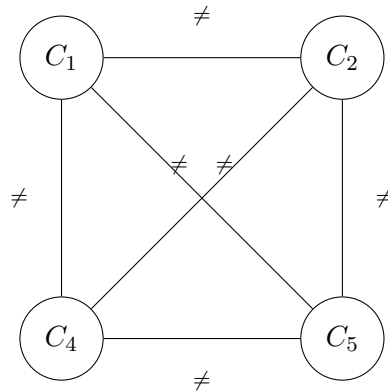
Question 1 • Model the problem as a constraint network $N = \langle X, D, C \rangle$. For each course C_i , introduce a variable $slot_i$ (the time slot assigned to C_i) and a variable $room_i$ (the room assigned to C_i). Give the domain of each variable, and state in words (then formally) the constraints induced by : (a) a teacher cannot teach two courses at the same time, (b) a student group cannot attend two courses at the same time, and (c) a room cannot host two courses at the same time, and must have enough capacity.

Question 2 • Draw the constraint graph : one node per course, and an edge between two courses whenever a constraint from Question 1 links them (label each edge with the reason : same teacher, shared group, or possible room conflict).

Question 3 • What is the size of the search space, i.e. the number of candidate assignments of $(slot_i, room_i)$ pairs to the 6 courses, before any constraint is taken into account ?

Exercise 2

To keep the manual propagation tractable, we now focus on the four courses taught in the L3A group's morning, restricted to the three Monday slots $\{1, 2, 3\}$: C_1, C_2, C_4, C_5 , all with domain $D = \{1, 2, 3\}$ for their *slot* variable. Recall that C_1 and no other course among these four shares a teacher with another ; C_4 shares group L3A with C_1, C_2, C_5 (and group L3B, ignored here) ; all four courses share group L3A, so *every* pair among $\{C_1, C_2, C_4, C_5\}$ must get a different slot. Suppose, in addition, a scheduling constraint has already pruned the domains to : $D(C_1) = \{1\}$, $D(C_2) = \{1, 2\}$, $D(C_4) = \{1, 2, 3\}$, $D(C_5) = \{2, 3\}$.



Question 1 • Write the initial propagation queue (list of arcs) that AC-3 will start with. Remember to include **both directions** for each constraint, e.g. for $slot_{C_1} \neq slot_{C_2}$, include $\overrightarrow{C_1 \neq C_2}$ and $\overrightarrow{C_2 \neq C_1}$.

Question 2 • Using the AC-3 algorithm, manually process the propagation queue step by step. For each step :

- Indicate which arc is being checked.
- Show any changes made to the domains of the variables.
- If a domain changes, add the necessary arcs back to the queue.

Question 3 • At the end of the process, write the final domains of all four variables.

Question 4 • Was the constraint network arc-consistent after running AC-3 ? Does arc-consistency alone guarantee that a full timetable for $\{C_1, C_2, C_4, C_5\}$ exists ? Explain briefly.

Exercise 3

We now compare search strategies on the same four variables C_1, C_2, C_4, C_5 , but starting from the *original* domains $D = \{1, 2, 3\}$ for all of them (i.e. before the pruning of Exercise 2), under the ordering C_1, C_2, C_4, C_5 and values tried in increasing order. Recall that every pair among the four courses is constrained by $\neq$ (they all share group L3A).

Question 1 • Run **chronological Backtracking (BT)** from the empty assignment. Draw the search tree until the first solution is found, showing every failed attempt and the backtracks it causes.

Question 2 • Run **Forward Checking (FC)** on the same instance. At each assignment, show how the domains of the future variables are filtered, and indicate at which point (if any) FC detects a failure earlier than BT did.

Question 3 • Run **MAC** (Maintaining Arc Consistency) on the same instance. Show that MAC detects the same failure as FC even earlier (or explain why, on this particular instance, it does not).

Question 4 • Generalize : with 4 courses that must pairwise get different slots and only 3 slots available, what can you conclude *without any search at all* ? Which level of consistency would catch this immediately, and how does it relate to the pigeonhole principle ?

Exercise 4

We now turn the timetabling CSP into an optimization problem (COP), and discuss redundant constraints, as is often done in practice to speed up solving.

Question 1 • Propose an objective function for the full 6-course instance of the introduction, e.g. minimizing the *makespan* (the index of the last used slot) or minimizing the total number of “holes” (idle slots between two courses) in each group’s daily schedule. Write it formally in terms of the $slot_i$ variables.

Question 2 • Rooms R1 and R2 are *not* interchangeable (they differ in capacity), but suppose the department later opens a third, identical room R2’ with the same capacity as R2. In pairs, discuss whether adding a symmetry-breaking constraint such as “a course assigned to {R2, R2’} always prefers R2 over R2’ when both are free” is beneficial for solving time. Justify your answer in a short report.

Question 3 • In pairs, discuss whether adding the following *implied* (redundant) constraint helps propagation : “at least n_{group} slots must be reserved for group L3A’s courses”, where n_{group} is the number of courses attended by L3A. Compare this to the redundant constraint $\text{ticks}[m-1] \geq m(m-1)/2$ sometimes added to Golomb ruler models : what role does a redundant constraint play in a CP model, and when can it backfire ?

Exercise 5

This assignment is to be completed in pairs, and the report must be submitted jointly.

Question 1 • Model the general n -course timetabling problem on paper as a constraint network $N = \langle X, D, C \rangle$ (variables $slot_i$ and $room_i$ for every course, as in Exercise 1, but now for an arbitrary set of courses, teachers, groups and rooms given as input data).

Below is a skeleton Choco-Solver model for this problem. Complete it and use it to answer the questions that follow.

Question 2 • Complete TODO 1 and TODO 2, and modify the code to enumerate **all** solutions for the reduced instance of Exercise 2 (C_1, C_2, C_4, C_5 with domain $\{1, 2, 3\}$). Does the number of solutions found match your search-tree analysis from Exercise 3 ?

Question 3 • Complete TODO 3 and TODO 4 and solve the full 6-course, 2-room instance given in the introduction. Report the timetable found.

Question 4 • Scale up : the department actually has 12 courses, 5 teachers, 4 student groups and 3 rooms, spread over 10 slots (a full week). Adapt your code so that it reads the courses/teachers/groups/rooms from arrays (or a small data file) instead of being hard-coded, and solve this larger instance. Report the solving time and, if no solution is found within a reasonable time limit, explain which part of the model you would revise first (search heuristic, redundant constraints, decomposition...).

Timetabling with teacher preferences

Some teachers who teach two courses in the same week would rather teach them in a fixed order (e.g. T_1 prefers to teach C_1 *before* C_3 , so that the lecture on Algorithmique precedes its application in Réseaux), while others need a minimum rest gap between their two courses. Figure 1 shows the preference constraints for our 6-course instance.



FIGURE 1 – Preference constraints : T_1 wants $slot_{C_1} < slot_{C_3}$; T_2 wants $|slot_{C_2} - slot_{C_6}| \geq 2$.

Question 5 • Revise the `Timetabling.java` model to add these two preference constraints (a strict precedence between C_1 and C_3 , and a minimum gap between C_2 and C_6), and re-solve the full 6-course instance. Does a solution still exist? If several preferences of this kind become mutually incompatible, how would you turn them into *soft* constraints instead, so that the solver returns the timetable violating the fewest preferences?